

Report for Distributed Programming For Web IoT And Mobile Systems

Jonathan Colombo

June 2026

1 Introduction

The goal of the proposed project is to design and implement a distributed system for managing a railway line and ticketing operations. The system provides a unified control panel for administrators and a customer-facing interface for purchasing tickets, viewing train schedules, and receiving notifications.

The project adopts a microservices architecture, where each service is responsible for a specific domain. Services communicate through REST APIs and, where required, through MQTT for asynchronous notifications. The entire system is containerized using Docker.

2 System Architecture

The system is composed of multiple independent microservices. Each service exposes its own API and interacts with a dedicated database schema. The architecture follows a modular design to ensure scalability, maintainability, and fault isolation.

The main components are:

- Users Service
- Authentication Service
- Trains Service
- Tickets Service
- Notifications Service
- MySQL Database
- Mosquitto MQTT Broker
- Frontend Web Application

Each service runs inside a dedicated Docker container. The services are orchestrated through Docker Compose, which ensures correct startup order and dependency management.

3 System Architecture Diagram

The following diagram provides a high-level overview of the system architecture, showing the microservices, the database layer, the MQTT broker, and the frontend application.

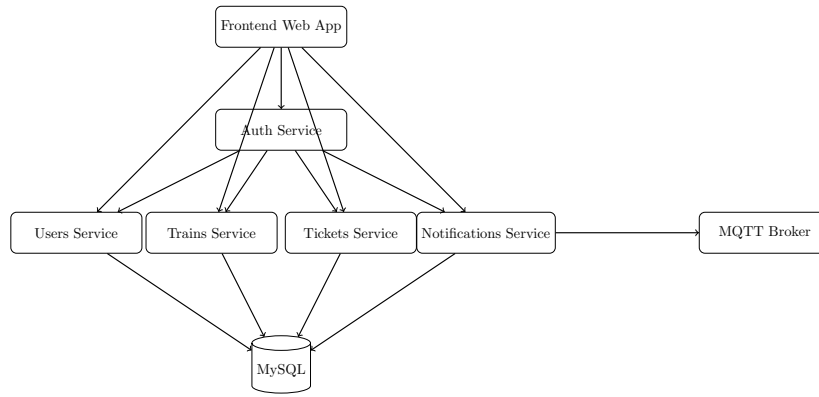


Figure 1: High-level architecture of the distributed railway management system.

4 Microservices Description

4.1 Users Service

The Users Service manages user registration and stores user information such as username, email, fiscal code, telephone number, and role. It exposes REST endpoints for creating and retrieving users. A startup script automatically seeds two demo users (admin and customer) for testing purposes.

4.2 Authentication Service

The Authentication Service handles login and JWT token generation. It validates user credentials by querying the Users Service API. Tokens are signed using a shared secret and are required to access protected endpoints in other services.

4.3 Trains Service

The Trains Service manages all data related to trains, including schedules, routes, and availability. It exposes endpoints for retrieving train information and is used by both the frontend and the Tickets Service.

4.4 Tickets Service

The Tickets Service handles ticket purchases and cancellations. It interacts with the Trains Service to verify availability and with the Authentication Service to validate user identity. Purchased tickets are stored in a dedicated database schema.

4.5 Notifications Service

The Notifications Service manages user subscriptions to specific trains. When a train status changes, the service publishes notifications through the Mosquitto MQTT broker. Customers subscribed to a train receive updates in real time.

5 Database Layer

The system uses a MySQL database with multiple schemas:

- **identity_users**: used by Users Service and Authentication Service
- **trains_db**: used by Trains Service
- **tickets_db**: used by Tickets Service
- **identity_subscriptions**: used by Notifications Service

Each service interacts only with its own schema, ensuring data isolation and reducing coupling. A single database instance hosting multiple schemas has been adopted to optimize resource utilization within the development environment. In a production scenario, each microservice would ideally rely on its own dedicated and independent database instance.

6 Communication Model

6.1 REST Communication

All services expose REST APIs for synchronous communication. The frontend interacts with the backend exclusively through these APIs.

6.2 MQTT Communication

The Notifications Service uses MQTT for asynchronous communication. The Mosquitto broker handles message distribution. This model is suitable for real-time updates such as train delays or cancellations.

7 Frontend Application

The frontend is implemented as a web application and provides:

- Login and authentication
- Train schedule visualization
- Ticket purchase interface
- Subscription management
- Real-time notifications

The frontend communicates with all backend services through environment-configured API URLs.

8 Security Considerations

Security is a fundamental aspect of the system, especially due to the distributed nature of the architecture and the presence of user authentication, personal data, and ticketing operations. The system adopts several mechanisms to ensure confidentiality, integrity, and isolation across services.

8.1 JWT-Based Authentication

Authentication is implemented using JSON Web Tokens (JWT). When a user logs in, the Authentication Service validates the credentials and issues a signed token containing both standard and custom claims. The system uses the `RegisteredClaims` structure, which includes:

- **sub**: the user identifier
- **role**: the user's role (admin or customer)
- **iat**: issued-at timestamp
- **nbf**: not-before timestamp
- **exp**: expiration timestamp (15 minutes)
- **jti**: a unique token identifier

Tokens are signed using the HS256 algorithm and a shared secret stored in the `AUTH_SECRET` environment variable. All microservices validate incoming requests by parsing the token, verifying its signature, and extracting the claims. If the token is valid, the middleware injects the `userID` and `role` into the request context, allowing downstream handlers to enforce authorization rules.

This approach ensures:

- stateless authentication without server-side session storage
- tamper-proof claims due to cryptographic signing
- short-lived tokens that reduce the impact of credential leakage
- clear separation between authentication (Auth Service) and authorization (individual microservices)

Clients must include the token in the `Authorization: Bearer <token>` header when accessing protected endpoints.

8.2 Service Isolation

Each microservice is isolated both logically and at the database level. No service has direct access to another service's internal data. Communication occurs exclusively through REST APIs, ensuring:

- clear separation of responsibilities
- reduced attack surface
- prevention of unauthorized cross-service access

8.3 Database Isolation

The MySQL instance hosts multiple schemas, one per service. Each service connects only to its own schema using dedicated credentials. This prevents:

- accidental data corruption
- unauthorized cross-service queries
- privilege escalation

8.4 MQTT Security

The Notifications Service publishes messages to the Mosquitto broker. Although the current implementation uses an open configuration for simplicity, the architecture supports:

- authenticated MQTT clients
- TLS encryption
- topic-level access control

These enhancements can be integrated in future iterations.

9 Deployment and Execution

The entire system is containerized using Docker. To start all services, the following command is sufficient:

```
docker compose up --build
```

Docker Compose builds the images, initializes the databases, starts the MQTT broker, and launches all microservices and the frontend. Two demo users are automatically created at startup.

10 Testing

The system can be tested through the frontend interface. The following demo accounts are available:

Admin

- Email: admin@example.com
- Password: Password123!

Customer

- Email: customer@example.com
- Password: Password123!

These accounts allow testing of all system functionalities, including ticket purchases and subscription to notifications.

11 User Experience and Interface Overview

The system provides a complete graphical interface that allows both administrators and customers to interact with the platform without requiring direct access to backend APIs. This section describes how the main functionalities can be tested through the web interface.

11.1 Customer Experience

Customers can log into the system using their credentials and access the following features:

- **View train schedules:** customers can browse all available itineraries and inspect departure times, destinations, and seat availability.
- **Purchase tickets:** the interface allows selecting a train and completing the purchase with a single action.

- **Manage subscriptions:** customers may subscribe to specific itineraries in order to receive notifications when an administrator modifies them.
- **Real-time notifications:** when a subscribed itinerary is updated, the customer receives a visual notification directly in the interface.

Notifications appear as temporary banners lasting approximately five seconds, ensuring that the user is immediately informed of relevant changes.

11.2 Administrator Experience

Administrators have access to additional management features:

- **Modify itineraries:** administrators can update train schedules, change departure, or adjust availability.
- **Trigger notifications:** any modification to an itinerary automatically generates a notification for all subscribed customers.
- **View system data:** administrators can inspect trains, tickets, and user information through dedicated panels.

11.3 Testing the Notification System

The notification system can be tested entirely through the graphical interface by following these steps:

1. Open two separate browser sessions or two different browsers.
2. Log in as a **customer** in the first session.
3. Log in as an **administrator** in the second session.
4. As the customer, navigate to the subscriptions section and subscribe to a specific itinerary.
5. As the administrator, modify the same itinerary (for example, by changing the departure station).
6. The customer interface will immediately display a real-time notification indicating that the subscribed itinerary has been updated.

This interaction demonstrates the integration between the Notifications Service, the MQTT broker, and the frontend interface. The customer receives the update without refreshing the page, confirming the correct functioning of the real-time communication layer.

12 Conclusions

The project demonstrates the design and implementation of a distributed microservices architecture for a railway management and ticketing system. The use of Docker ensures portability and reproducibility, while the modular design allows each service to evolve independently. The integration of MQTT enables real-time communication, enhancing the user experience.

The system is fully functional and provides a solid foundation for future extensions such as payment integration, advanced analytics, or mobile app support.